

Alpamayo-R1 VRAM 최적화 연구 보고서

날짜

2026-02-26

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

1. Alpamayo-R1 VRAM 사용량 분석

- 1.1 모델 구조
- 1.2 추론 시 VRAM 점유 비율
- 1.3 일반 LLM과의 KV Cache 비교
- 1.4 시각화

2. 스왑 오버헤드 분석

- 2.1 Vanilla Alpamayo (BF16, Unified Memory)
- 2.2 4-bit 양자화 + VRAM 제한 실험

3. 기존 오프로딩 프레임워크의 한계

- 3.1 device_map="auto" (HuggingFace Accelerate)
- 3.2 수동 오프로딩 (max_memory 제약)
- 3.3 커스텀 구현이 필요한 이유

4. Demand Layering 적용 결과

- 4.1 Demand Layering 논문 (RTSS 2022)
- 4.2 실험 결과
- 4.3 시간 구성 분석

5. 스왑 방법론 비교

- 실행시간 산출 근거
- 5.1 방법 1: 레이어 단위 동기 스왑 (Synchronous On-Demand)
- 5.2 방법 2: 최적 청크 전송 (2-8MB Chunked Transfer)
- 5.3 방법 3: 모듈 단위 스왑 → 불가능
- 5.4 교수님 아이디어: 2-Stage 스와핑 분석
- 5.5 방법 4: 비동기 2-Stream 파이프라인 (Prefetch + Free)

6. 실행시간 비교 종합

7. WSL2 환경 오버헤드

- 7.1 측정된 WSL2 오버헤드
- 7.2 의미

참고 문헌

Alpamayo-R1 VRAM 최적화 연구 보고서

작성일: 2026-02-26 환경: NVIDIA GeForce RTX 3080 Ti (12 GB VRAM), WSL2, PCIe Gen3 x16

1. Alpamayo-R1 VRAM 사용량 분석

1.1 모델 구조

Alpamayo-R1-10B은 11.08B 파라미터, BF16 기준 약 22.16GB(실측 Peak 21.52GB)의 VLA(Vision-Language-Action) 모델이다.

모듈	레이어 수	크기 (BF16)	비율
Vision Encoder	—	1.15 GB	5.2%
VLM (Qwen3-VL-8B)	36	15.17 GB	68.5%
VLM LM Head	—	1.28 GB	5.8%
Expert (Trajectory Decoder)	36	4.56 GB	20.6%
합계	—	~22.16 GB	100%

1.2 추론 시 VRAM 점유 비율

분류	구성요소	크기	비율
정적 (모델 파라미터)	VLM Language Model	15.17 GB	63.7%
	Vision Encoder	1.15 GB	4.8%
	VLM LM Head	1.28 GB	5.4%
	Expert (Decoder)	4.56 GB	19.1%
	소계	22.16 GB	93.0%
동적 (추론 중)	KV Cache (~4,470 토큰)	0.56 GB	2.4%
	Activation + Overhead	1.10 GB	4.6%
	소계	1.66 GB	7.0%
합계		~23.82 GB	100%
측정 Peak		21.52 GB	

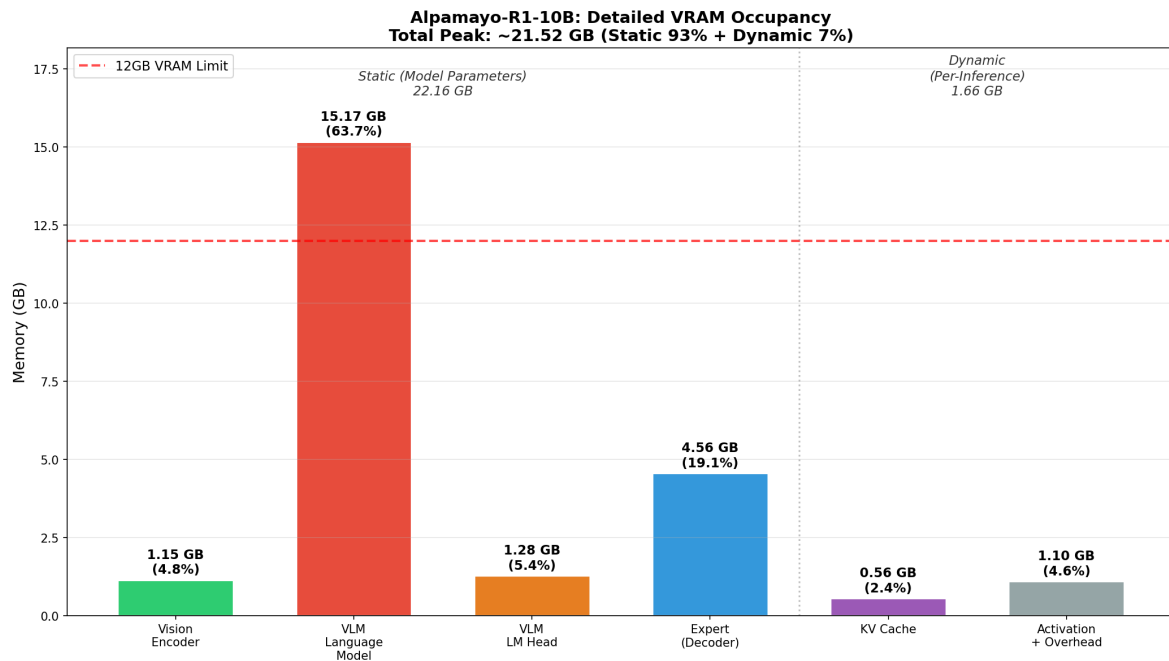
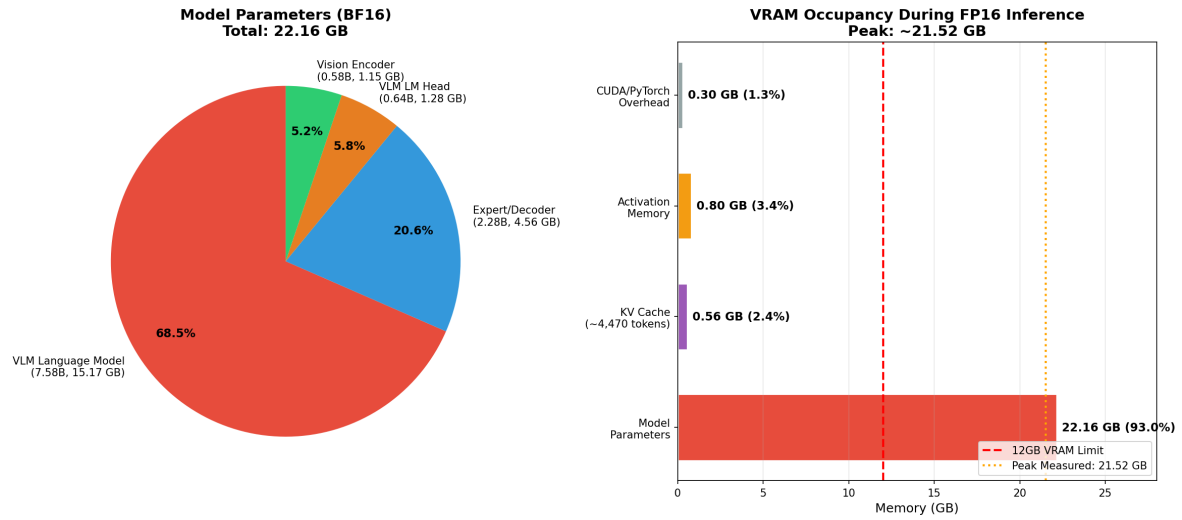
1.3 일반 LLM과의 KV Cache 비교

일반 LLM에서 KV Cache는 긴 컨텍스트에서 총 메모리의 **30-50%+** 를 차지하여 메모리 병목의 핵심 요인이다. Alpamayo는 이와 근본적으로 다르다:

	일반 LLM (예: Llama-70B, 4K 컨텍스트)	Alpamayo-R1
KV Cache 비중	총 메모리의 30-50%+	총 메모리의 2.4%
원인	긴 컨텍스트 + Full MHA	GQA + 짧은 CoT (~15토큰)
KV Cache 크기	수~수십 GB	0.56 GB
메모리 병목	KV Cache + 파라미터	파라미터 단독 (93%)

핵심 시사점: Alpamayo에서는 KV Cache 최적화가 아닌, **파라미터 자체의 메모리 최적화**가 핵심 과제이다. 특히 VLM 단독 15.17GB > 12GB VRAM이므로, 레이어 단위 파라미터 스와핑이 필수적이다.

1.4 시각화



2. 스왑 오버헤드 분석

2.1 Vanilla Alpamayo (BF16, Unified Memory)

실험 환경: RTX 3080 Ti 12GB, WSL2, PyTorch 2.8.0+cu128

모델 전체(~22GB)를 GPU에 로드하면 12GB VRAM을 초과하는 부분이 CUDA Unified Memory를 통해 자동으로 CPU RAM에 배치된다.

실행 결과: 273.79s (이론 최적 ~5s 대비 55배 느림)

느린 이유 분석:

1. **On-Demand 페이지 폴트**: 모델 ~22GB > 12GB VRAM → Unified Memory가 4KB~2MB 단위로 page fault 발생
2. **PCIe 대역폭 효율 저하**: 소규모 페이지 단위 전송으로 유효 대역폭이 이론치 대비 크게 감소
3. **매 토큰 전체 VLM 읽기**: 토큰 생성마다 VLM 15.17GB 파라미터 전체를 읽어야 함 (256토큰 × 15.17GB)
4. **대역폭 격차**: VRAM 내부 912 GB/s vs PCIe Gen3 8.5 GB/s = **107배 격차**

이론 최적값 산출 (VRAM bandwidth-bound, 스왑 없는 경우):

단계	읽기량	반복	시간
VLM (토큰 생성)	16.45 GB/토큰	256회	4.62s
Expert (디퓨전)	4.56 GB/스텝	10회	0.05s
Vision + KV Cache 등			~0.3s
합계			≈ 5.0s

연산 시간(0.44ms/토큰)은 읽기 시간(18ms/토큰)의 2.4%로 무시 가능. **Memory-bandwidth-bound** 워크로드.

2.2 4-bit 양자화 + VRAM 제한 실험

4-bit 양자화 모델(dwko/Alpamayo-R1-10B-4bit)을 사용하여 VRAM 가용량에 따른 성능 변화를 측정했다. dummy 텐서로 VRAM을 선점하여 사용 가능 VRAM을 제한하는 방식이다.

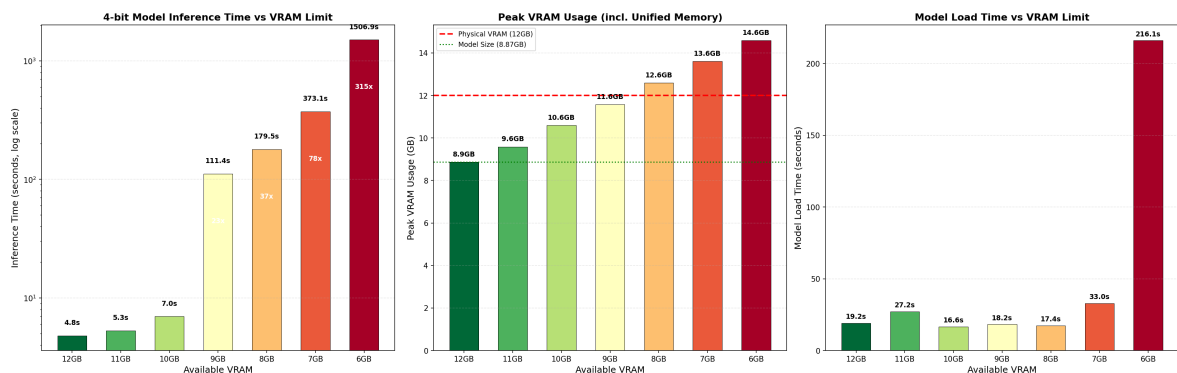
- **4-bit 모델 크기**: 8.87 GB (Peak VRAM, 12GB 제한 시)

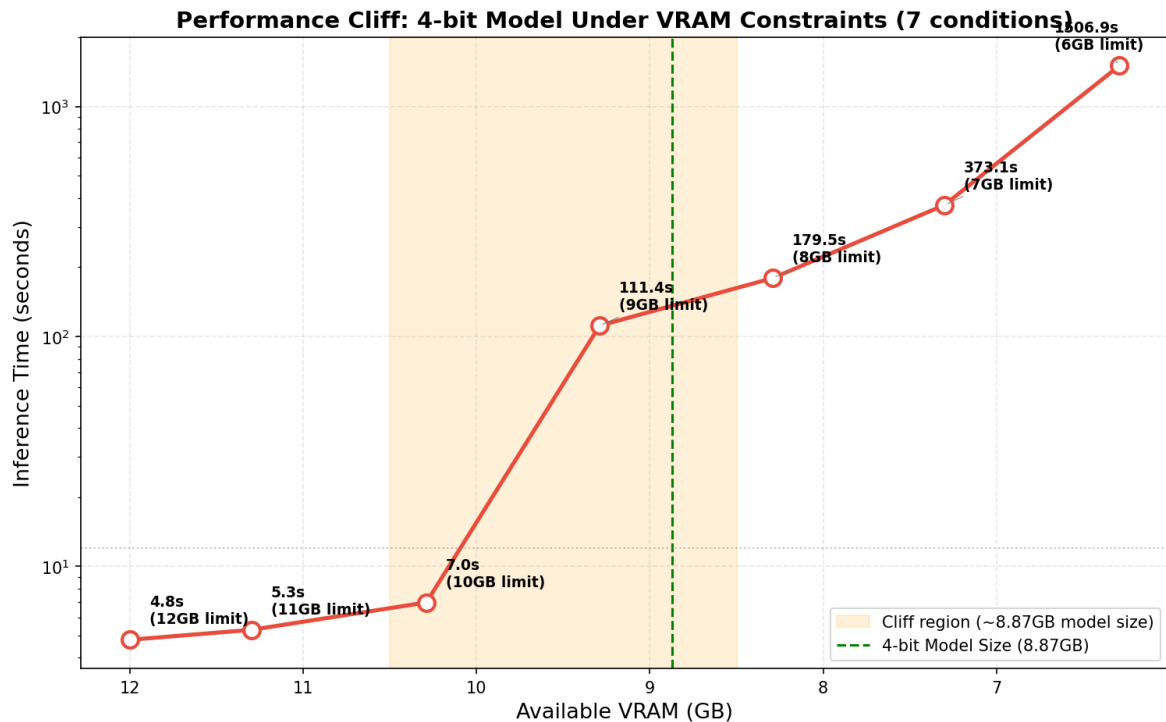
실험 결과:

VRAM 제한	Dummy 크기	가용 VRAM	추론 시간	Peak VRAM	비고
12 GB	0 GB	12.00 GB	4.79s	8.87 GB	Baseline
11 GB	0.7 GB	11.30 GB	5.29s	9.59 GB	
10 GB	1.7 GB	10.29 GB	6.96s	10.59 GB	
9 GB	2.7 GB	9.29 GB	111.36s	11.58 GB	Performance Cliff
8 GB	3.7 GB	8.29 GB	179.46s	12.59 GB	
7 GB	4.7 GB	7.30 GB	373.05s	13.61 GB	재측정 완료
6 GB	5.7 GB	6.29 GB	1506.88s	14.59 GB	

Performance Cliff 분석:

- 10GB → 9GB 구간에서 16배 급감 (6.96s → 111.36s)
- 가용 VRAM이 모델 크기(8.87GB)를 밑돌기 시작하면 Unified Memory 스왑 발생
- 스왑 발생 시 성능이 불연속적으로 급락하는 cliff 현상





3. 기존 오프로딩 프레임워크의 한계

3.1 device_map="auto" (HuggingFace Accelerate)

VRAM 부족 시 가장 먼저 시도하는 표준 방법은 HuggingFace Accelerate의 `device_map="auto"` 이다.

```
model = AutoModelForCausalLM.from_pretrained("model_name", device_map="auto")
```

동작 방식: 1. 모델 로드 시 VRAM 용량 확인 2. VRAM에 들어가는 레이어는 GPU에 배치 3. 초과분은 CPU RAM에 배치 (그래도 부족하면 디스크) 4. 추론 시 CPU에 있는 레이어를 실행 차례에 GPU로 이동 → 완료 후 CPU로 반환

실험 결과: 추론 실패

Accelerate가 Alpamayo-R1에 자동 배치한 결과:

컴포넌트	배치 위치
Vision Encoder + VLM 레이어 0-16 (17개)	GPU (4.49B, 40.6%)
VLM 레이어 17-35 + LM Head + Expert 전체	CPU/Disk (6.58B, 59.4%)

모델 로드는 성공 (VRAM 8.99GB), 그러나 추론 시 CUDA device-side assert 발생.

실패 원인 — `fuse_traj_tokens()` 의 디바이스 불일치:

Alpamayo-R1의 `fuse_traj_tokens()` 메서드는 VLM 출력 텐서와 trajectory 토큰 테이블을 `masked_scatter` 로 결합한다. 이 연산은 입력 텐서와 소스 텐서가 **동일한 디바이스**에 있어야 한다. `device_map="auto"` 로 레이어가 GPU/CPU에 분산 배치되면 이 조건을 만족하지 못한다.

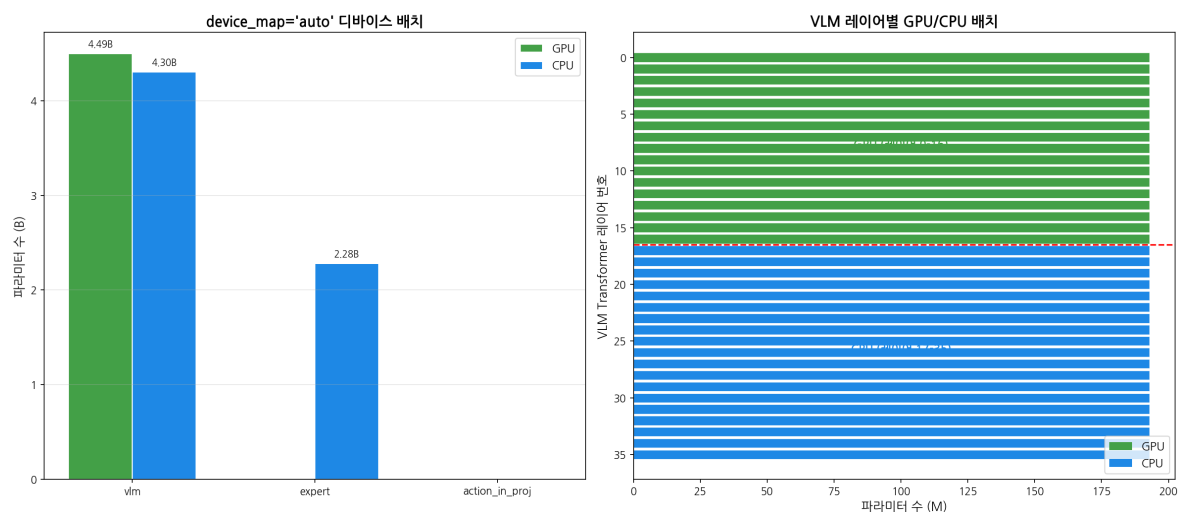
3.2 수동 오프로딩 (max_memory 제약)

`max_memory` 파라미터로 GPU 10GB + CPU 10GB + Disk 구성을 시도했으나, 동일한 이유로 추론 실패. 커스텀 토큰 처리 파이프라인이 Accelerate의 자동 디바이스 전환과 호환되지 않는다.

3.3 커스텀 구현이 필요한 이유

문제	설명
<code>fuse_traj_tokens()</code> 비호환	<code>masked_scatter</code> 가 디바이스 분산과 충돌
KV Cache 공유	Expert가 VLM의 <code>past_key_values</code> 를 직접 참조 → 별도 디바이스 불가
<code>generate()</code> 내부 가정	HuggingFace의 <code>generate()</code> 는 모든 모듈이 동일 디바이스에 있다고 가정

결론: 기존 프레임워크(`device_map` , Accelerate)의 자동 오프로딩은 Alpamayo에서 동작하지 않는다. 레이어 단위 커스텀 오프로딩 구현이 필수적이며, 이것이 본 연구의 출발점이다.



4. Demand Layering 적용 결과

4.1 Demand Layering 논문 (RTSS 2022)

RTSS 2022 논문 “Demand Layering for Real-Time DNN Inference with Minimized Memory Usage”는 DNN 추론 시 모델 파라미터를 레이어 단위로 로딩/실행하는 기법을 제안한다.

원 논문의 3-Stage Synchronous Pipeline:

```
Layer i:    [ Read_i  ][ Copy_i  ][ Kernel_i ]
Layer i+1:      [ Read_i+1 ][ Copy_i+1 ][ Kernel_i+1 ]
Layer i+2:          [ Read_i+2 ][ Copy_i+2 ][ Kernel_i+2 ]
```

- **Read:** SSD → CPU staging buffer (DMA)
- **Copy:** CPU staging → GPU 메모리 (PCIe)
- **Kernel:** GPU에서 레이어 실행

원 논문 vs 우리 환경:

	원 논문 (RTSS 2022)	본 연구
GPU	iGPU (통합 메모리)	dGPU (RTX 3080 Ti, 12GB)
스토리지	NVMe SSD	CPU RAM
파이프라인	3-Stage (Read→Copy→Kernel)	Sequential (H2D→Execute→Free)
대상 모델	단일 DNN	VLA (Vision-Language-Action)

파라미터 오프로딩 추론에서는 D2H(GPU→CPU 복사)가 불필요하다. CPU에 파라미터 원본이 보존되어 있으므로, 사용 완료된 레이어는 GPU 메모리를 해제(free)하기만 하면 된다.

4.2 실험 결과

구현: `research/07-demand-layering-no-d2h/demand_layering_no_d2h.py` - VLM 36개 레이어 중 30개를 CPU로 오프로드, 6개는 GPU 상주 - `register_forward_pre_hook()` / `register_forward_hook()` 으로 동적 로드/Free - D2H 제거: `module.to("cpu")` 대신 CPU 원본 참조 복원 + GPU 메모리 해제

항목	값
추론 시간	43.38s
Peak VRAM	11.03 GB
GPU 상주	9.86 GB
H2D 전송	450회, 총 24.23s (평균 53.83 ms/layer)
GPU Free	450회, 총 1.05s (평균 2.32 ms/layer)
vs Baseline (273.79s)	6.31x 빠름

4.3 시간 구성 분석

추론 시간 43.38s의 구성:

H2D 전송: 24.23s (55.9%)
계산: ~18.1s (41.7%)
GPU Free: 1.05s (2.4%)

합계: 43.38s

핵심 관찰:

1. H2D 전송이 추론 시간의 56% — PCIe 대역폭이 여전히 병목
2. GPU Free는 2.4%로 무시 가능 (평균 2.32ms/layer) — D2H 대비 67배 빠름
3. 동기식 전송으로 PCIe 유휴 시간 多 — 비동기 오버랩으로 추가 개선 가능
4. Per-layer: H2D 53.83ms + Compute 40.2ms + Free 2.32ms = ~96ms

5. 스왑 방법론 비교

실행시간 산출 근거

- VLM 36레이어 중 30개 오프로드 × 15 VLM passes = **450 transfers**
- 레이어당 크기: ~386 MB (BF16), ~0.193 GB
- Per-layer H2D (실측): 53.83 ms
- Per-layer compute (실측): ~40.2 ms (18.1s ÷ 450)
- Per-layer Free (실측): 2.32 ms

5.1 방법 1: 레이어 단위 동기 스왑 (Synchronous On-Demand)

개념: 개별 Transformer 레이어를 forward hook으로 GPU에 로드 → 실행 → GPU 해제. CPU 원본은 유지되므로 D2H(GPU→CPU 복사)는 수행하지 않는다.

```
Layer N: [H2D 54ms][Compute 40ms][Free 2ms]
Layer N+1: [H2D 54ms][Compute 40ms][Free 2ms]
→ 레이어당 ~96ms, 전체: 43.38s
```

성능: 43.38s (실측, vs Baseline 273.79s → 6.31x 가속)

구현 상태: 완료 ([research/07-demand-layering-no-d2h/demand_layering_no_d2h.py](#))

5.2 방법 2: 최적 청크 전송 (2-8MB Chunked Transfer)

개념: 레이어를 2-8MB 청크로 분할 + Pinned Memory로 PCIe 대역폭 극대화.

```
현재: [단일 전송], Pageable 7.77 GB/s
개선: 2-8MB 청크, Pinned 12.4 GB/s (1.6x)
```

청크 크기	유효 대역폭	비고
2-8 MB	12.16-12.41 GB/s	최적 구간
512 MB (단일)	7.77 GB/s	기준
PCIe Gen3 이론	15.75 GB/s	실측 79% 활용

성능: H2D 24.23s / 1.6 ≈ 15s + compute 18.1s + free 1.0s ≈ ~34s

구현 상태: 벤치마크 완료 ([research/03-swap-optimization/](#))

5.3 방법 3: 모듈 단위 스왑 → 불가능

개념: Vision / VLM / Expert를 모듈 통째로 스왑.

검증 결과:

```
Vision Encoder: 1.15 GB → ✓ GPU 적재 가능
VLM:            15.17 GB → ✗ 12GB 초과, 모듈 단위 불가
Expert:         4.56 GB → ✓ GPU 적재 가능
```

VLM 단독 15.17GB > 12GB VRAM — 순수 모듈 단위 스왑은 불가능.

5.4 교수님 아이디어: 2-Stage 스와핑 분석

교수님이 제안하신 비전-프리필과 디코딩을 2-stage로 나누어 VRAM을 절감하는 아이디어에 대한 분석:

A. 모듈 단위 2-Stage

- Stage 1: Vision + VLM → GPU
- Stage 2: Expert → GPU
- 문제: VLM 단독 15.17GB > 12GB → 불가능

B. 레이어 단위 2-Stage (수정 적용)

- Stage 1: VLM 레이어별 스왑으로 prefill + token generation 수행 (Expert는 CPU 대기)
- Stage 2: Expert 전체를 GPU에 로드 (4.56GB, 적재 가능) → diffusion 실행

이론 시간: Stage 1 (VLM 레이어별 스왑) + Stage 2 (Expert 로드 0.5s + diffusion ~0.05s)

결론: 레이어 단위 2-stage는 유효하며, 방법 1의 현재 구현이 이미 이 구조에 해당한다. (VLM은 레이어별 스왑, Expert는 GPU 상주로 처리)

5.5 방법 4: 비동기 2-Stream 파이프라인 (Prefetch + Free)

개념: 2개의 CUDA 스트림으로 계산과 H2D prefetch를 비동기 오버랩. 방법 1이 동기식(H2D → Compute → Free 순차)인 반면, 방법 4는 다음 레이어의 H2D를 현재 레이어의 Compute와 겹친다.

시간 →
Compute Stream: [Layer N 계산] [Layer N+1 계산] [Layer N+2 계산]
H2D Stream: [Layer N+1 로드] [Layer N+2 로드] [Layer N+3 로드]
 + Layer N-1 free + Layer N free + Layer N+1 free

이론 시간 산출 (실측 기반):

시나리오	Per-layer	450 layers	총 시간
기본 비동기 (Pageable)	$\max(53.83\text{ms}, 40.2\text{ms}) = 53.83\text{ms}$	$450 \times 53.83\text{ms}$	~24s
+ Chunked (12.4 GB/s)	$\max(31\text{ms}, 40.2\text{ms}) = 40.2\text{ms}$	$450 \times 40.2\text{ms}$	~18s

실험 결과 (Experiment 08)

구현: `research/08-async-pipeline/async_pipeline.py`

Variant A — Pageable Async (`non_blocking=True`):

항목	값
추론 시간	44.16s — 개선 없음
원인	<code>non_blocking=True</code> + pageable memory는 CUDA 내부에서 bounce buffer staging이 발생하여 사실상 동기 전송

Pageable 메모리에서 `non_blocking=True` 는 CUDA가 내부적으로 pinned bounce buffer로 복사 후 DMA를 수행하는데, 이 CPU-side 복사가 블로킹되어 비동기 효과가 없다.

Variant C — Staged Pinned Buffer (Double-Buffered):

항목	값
추론 시간	38.45s (1.13x vs baseline 43.38s)
Peak VRAM	11.38 GB
Pinned Host Memory	~772 MB (double-buffered staging)
DMA 오버랩	성공 — wait_dma 평균 0.05ms (DMA가 compute 중 완료)
CPU staging 시간	평균 44ms (pageable → pinned memcpy)
VLM passes	13회 (baseline 15회 — 토큰 생성 variance)

핵심 관찰:

- DMA 오버랩 검증 성공:** pinned memory 사용 시 DMA가 compute와 완전히 오버랩됨 (wait_dma 0.05ms)
- 병목 전이:** H2D DMA → **CPU staging memcpy (44ms)** 로 병목이 이동
- 전체 파라미터 pinning 불가:** VLM 오프로드 파라미터 ~11.6GB를 전부 pin하면 16GB RAM 한도 초과
- 실질 개선 13%:** 이론 예측(~24s) 대비 제한적 — CPU memcpy가 DMA 시간을 대체

결론: 비동기 파이프라인 개념은 실험적으로 검증되었다. DMA는 pinned memory를 사용할 때 compute와 진정으로 오버랩된다. 그러나 메모리 제약 시스템(16GB RAM)에서는 CPU staging memcpy가 새로운 병목이 되어 실질 개선은 ~13%에 그친다. 전체 파라미터를 pinned memory에 올릴 수 있는 충분한 RAM(32GB+)이 있다면 이론 예측치(~24s)에 근접할 것으로 예상된다.

Variant D — Threaded `stage_and_prefetch` (Background Thread):

Variant C의 full-layer pinned double buffer 방식을 background thread에서 실행하는 방식이다. CPU memcpy(~44ms)를 background thread에서 실행하여 GPU compute와 오버랩한다. Python의 GIL은 C++ 확장인 `tensor.copy_()` 실행 중에는 해제되므로, CPU memcpy와 GPU compute가 진정으로 병렬 실행된다.

항목	값
per-pass	2.41s (Baseline 2.89s, Variant C 2.96s)
vs Baseline	약 1.20x per-pass 개선
Peak VRAM	11.08 GB (안전)
병목	DMA latency (~46ms) — memcpy 완료 후에야 DMA 시작

핵심 관찰:

1. **CPU memcpy 오버랩 성공:** background thread에서 CPU memcpy를 실행하여 GPU compute 시간과 겹침
2. **GIL 회피 확인:** `tensor.copy_()` 는 C++ 레벨에서 실행되어 GIL이 해제됨
3. **DMA latency 잔존:** memcpy가 완료된 후에야 DMA가 시작되므로 ~46ms DMA latency 는 오버랩되지 않음
4. **per-pass 기준 유의미한 개선:** Variant C(2.96s)보다 빠르고, Baseline(2.89s)보다도 빠름

Chunked Transfer 실험 — 실패:

chunk-level pipelining (4MB 청크, ring buffer 등)과 per-param interleaving (memcpy→DMA를 param별로 교차)을 시도했으나, 모두 **파라미터 corruption** 발생으로 실패했다.

시도	방법	결과
Chunk-level pipelining	4MB 청크 단위 ring buffer	파라미터 corruption
Per-param interleaving	param별 memcpy→DMA 교차	파라미터 corruption
순차 실행 (all memcpy → all DMA)	chunking 없음	정상 동작

실패 원인 분석: background thread에서 `with torch.cuda.stream()` 블록 안에서 CPU memcpy와 CUDA DMA를 혼합할 때, 데이터 정합성 문제가 발생한다. CUDA stream의 동기화 경계에서 CPU 측 버퍼 상태가 불확실해지기 때문이다. chunking 없이 순차적으로 (all memcpy → all DMA) 실행할 때만 정상 동작이 확인되었다.

결론: Variant D (threaded stage_and_prefetch)는 per-pass 기준 1.20x 개선을 달성했다. CPU memcpy의 오버랩은 성공했으나, DMA latency의 추가 오버랩(chunked transfer)은 데이터 정합성 문제로 불가능했다. 현재 환경(16GB RAM)에서의 실질적 최선 방법이다.

선행 연구 대비 차별점:

시스템	연도	H2D Prefetch	D2H 사용 이유	대상
SwapAdvisor (ASPLOS)	2020	O	Activation 저장 (학습)	학습
Demand Layering (RTSS)	2022	O	SSD writeback	RT 추론
FlexGen (ICML)	2023	O	KV Cache + Activation offload	배치 추론
PIPO	2025	O	KV Cache saving	소비자 추론
본 연구	2026	O	불필요 (free만)	VLA 추론

6. 실행시간 비교 종합

방법	추론 시간	vs Baseline (273.79s)	vs 방법1 (43.38s)	비고
Baseline (BF16 Unified Memory)	273.79s	1x	—	실측
방법 1 (동기 H2D + Free)	43.38s	6.31x	1x	실측
방법 2 (청크 전송)	~34s	8.1x	1.28x	이론 (Pinned + 2-8MB 청크)
방법 3 (모듈 단위)	N/A	—	—	VLM > 12GB, 불가
방법 4a (비동기, Pageable)	44.16s	6.21x	0.98x	실측 — non_blocking 무효
방법 4c (비동기, Staged Pinned)	38.45s	7.12x	1.13x	실측 — DMA 오버랩 성공
방법 4d (Threaded stage_and_prefetch)	per-pass 2.41s	—	1.20x per-pass	실측 — CPU memcpy 오버랩 성공
방법 4 이론 (전체 Pinned)	~24s	11.4x	1.81x	32GB+ RAM 필요
방법 4 + 2 (비동기 + 청크)	~18s	15.2x	2.41x	이론 최선
이론 최적 (스왑 없음)	~5s	55x	8.68x	24GB+ GPU 필요

실험 결과 요약: 방법 4c (Staged Pinned Buffer)로 **43.38s → 38.45s**, 13% 가속 실측. 방법 4d (Threaded)로 per-pass 기준 **2.89s → 2.41s**, 약 20% 가속 실측. DMA 오버랩은 성공하나, 16GB RAM 제약으로 CPU staging memcpy(44ms)가 새 병목. Chunked transfer는 데이터 정합성 문제로 불가, 순차 실행(all memcpy → all DMA)만 정상 동작. 32GB+ RAM에서 전체 파라미터 pinning이 가능하면 이론치 ~24s에 근접할 것으로 예상.

7. WSL2 환경 오버헤드

현재 실험 환경은 네이티브 Linux가 아닌 WSL2 위에서 동작한다.

7.1 측정된 WSL2 오버헤드

항목	WSL2 (현재)	네이티브 Linux (예상)	차이
Pageable D2H 대역폭	2.48 GB/s	~11 GB/s	4.5배 느림
Pinned D2H 대역폭	11.8 GB/s	~12+ GB/s	유사
Pageable H2D 대역폭	7.77 GB/s	~8+ GB/s	유사
per-transfer 고정 오버헤드	0.36 ms	~0.05 ms	7배
cudaMemPrefetchAsync(CPU)	미지원	지원	—

7.2 의미

- WSL2의 Pinned memory 대역폭은 네이티브와 유사 → **Pinned 기반 구현에서는 영향 적음**
- 방법 4 (비동기 파이프라인)는 Pinned memory를 사용하므로 WSL2 오버헤드 영향 최소
- 최적화 방향성(pinned + 비동기 + 레이어별) 탐색에는 WSL2에서도 유효
- 최종 성능 평가는 네이티브 Linux에서 수행하는 것이 바람직

참고 문헌

- Demand Layering — Bae et al., RTSS 2022 (arXiv:2210.04024)
- SwapAdvisor — Huang et al., ASPLOS 2020
- FlexGen — Sheng et al., ICML 2023 (arXiv:2303.06865)
- PIPO — 2025 (arXiv:2504.03664)
- vDNN — Rhu et al., 2016 (arXiv:1602.08124)